

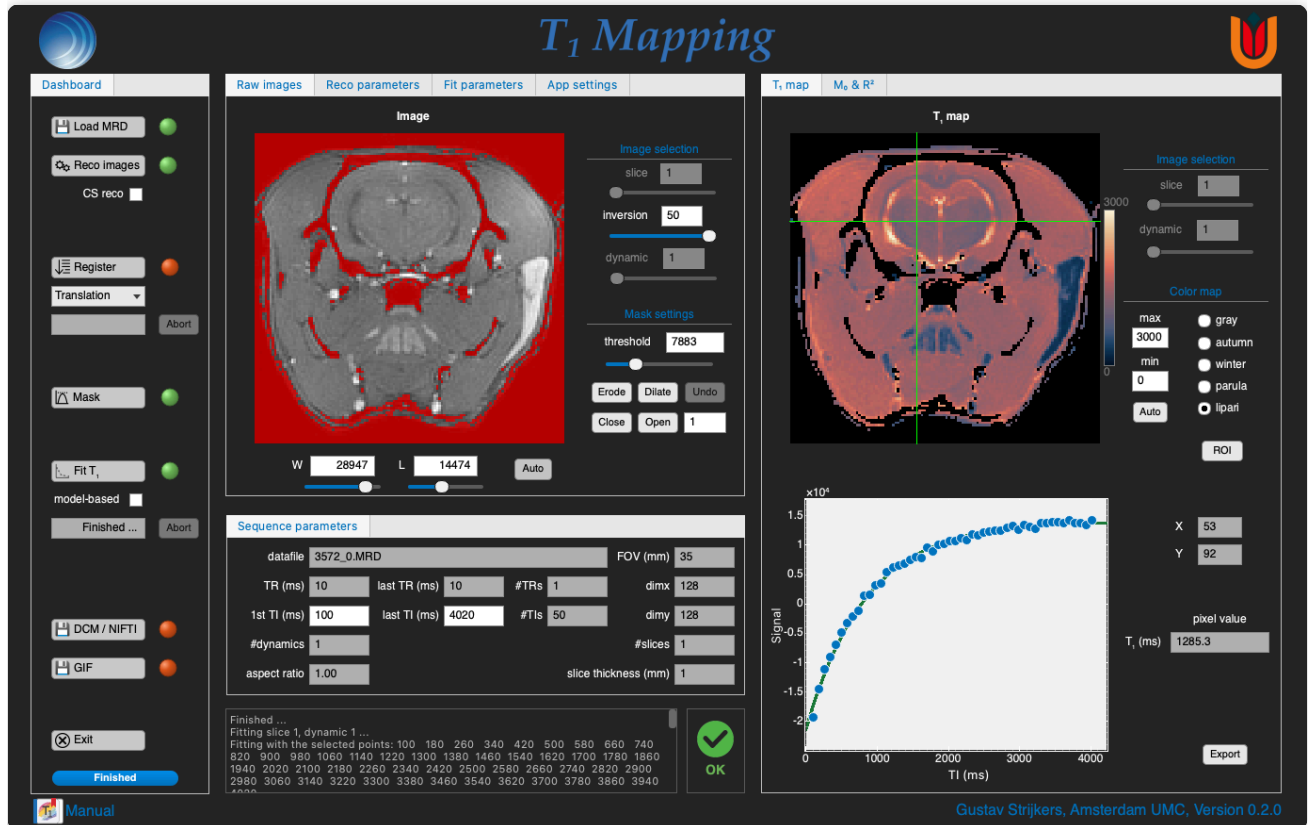


Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Installation
5. Running the software
6. Basic operation
7. Differences from the MATLAB app
8. Credits



1. Overview

T1mapp is an application with a graphical user interface to reconstruct T_1 maps from inversion recovery (Look-Locker), saturation recovery or variable flip angle experiments acquired with an MR Solutions preclinical MRI system. The images are reconstructed, registered, masked and fitted to T_1 , M_0 and R^2 maps, which can be exported as DICOM, NIfTI, BIDS and GIF, with the values of a region of interest as a CSV file.

This is the stand-alone version of T1mapp. It needs no MATLAB and no Python: everything it uses, elastix and dcm2niix included, is inside the application. Its window, controls and results are those of the MATLAB app T1mapp, which it follows; section 7 lists where the two differ.

2. Contact

T1mapp and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.striekers@amsterdamumc.nl

3. Disclaimer

This is a beta version. The software has been extensively tested using mouse and rat data acquired with an MR Solutions preclinical 7.0T/24cm system. However, this does not warrant that the functions contained in the program will meet your requirements or that the operation of the program will be uninterrupted or error-free. In case of questions or issues, please contact Gustav Strijkers.



4. Installation

macOS

Requirements: a Mac with Apple silicon (M1 or later), macOS 26 or later.

T1mapp comes as a disk image, `T1mapp-0.2.0-macOS-arm64.dmg`. Nothing else has to be installed: Python, elastix, dcm2niix and everything else the app uses are inside it.

1. Open the disk image and drag **T1mapp** onto the Applications folder in its window.
2. Open it from Applications. The app is not signed with an Apple Developer ID, so the first time macOS refuses it ("Apple could not verify ..."); click **Done**. Then open **System Settings > Privacy & Security**, scroll down to the line saying T1mapp was blocked, click **Open Anyway** and confirm. From then on it opens as any other app.

Instead of step 2, the same in Terminal, once:

```
xattr -dr com.apple.quarantine /Applications/T1mapp.app
```

The first start can take a little longer: macOS checks the app once.

Windows (provisional)

The Windows version has not been released yet. This section describes the installer as it is prepared, and will be corrected once it has been tested.

Requirements: Windows 10 or 11, 64-bit.

T1mapp comes as an installer, `T1mapp-0.2.0-setup.exe`. It installs the app for the current user (no administrator rights needed), with a Start-menu entry, an optional desktop shortcut and an uninstaller. Nothing else has to be installed.

BART (optional)

With BART, the Berkeley Advanced Reconstruction Toolbox (<https://mrirecon.github.io/bart/>), version 1.0.00 or newer, the compressed sensing reconstruction runs with BART, and the model-based T_1 fit is available. Without BART, or with an older version, the compressed sensing reconstruction runs with the app's own engine, based on sigpy, and the model-based fit is switched off. The message box says so at start. When BART is built with GPU support, T1mapp runs it on the GPU.

Installing BART on macOS with MacPorts

1. Install the Xcode command line tools:

```
xcode-select --install
```

2. Install MacPorts from <https://www.macports.org/>.

3. Install the packages:

```
sudo port install fftw-3-single gcc14 libpng openblas flock gmake coreutils 1  
ibblocksruntime
```



4. Download and unpack BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
```

5. Build and install:

```
export PATH=/opt/local/libexec/gnubin:$PATH
CC=gcc-mp-14 gmake
sudo gmake install
```

The compiler name must match the gcc version installed in step 3: gcc14 gives gcc-mp-14, gcc15 gives gcc-mp-15.

6. Check that it works: `bart version`

Installing BART on macOS with Homebrew

1. Install the Xcode command line tools: `xcode-select --install`

2. Install Homebrew:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

3. Install the packages BART needs:

```
brew install gcc libpng fftw openblas make llvm libomp
```

4. Download and unpack BART as above (step 4 of MacPorts).

5. Build and install:

```
CC=gcc-15 MACPORTS=0 gmake
sudo gmake install
```

The compiler name must match the version Homebrew installed; check it with `ls /opt/homebrew/bin/gcc-*`.

6. Check that it works: `bart version`

How T1mapp finds BART on macOS

`sudo make install` puts BART in `/usr/local/bin`, which is where the app looks first. It looks in these folders, in this order:

Folder

<code>/usr/local/bin</code>	where BART's own <code>make install</code> puts it
<code>/usr/bin</code>	
<code>/opt/homebrew/bin</code>	Homebrew's
<code>/opt/local/bin</code>	MacPorts'

An app opened from the Finder or the Dock does not see the settings of your shell (such as `TOOLBOX_PATH` or `PATH` in your shell profile). If your `bart` is in another folder, a link to it in `/usr/local/bin` lets the app find it. In Terminal, once (Terminal asks for your password):



```
sudo mkdir -p /usr/local/bin
sudo ln -s /path/to/bart/bart /usr/local/bin/bart
```

Installing BART on Windows 11 (provisional)

On Windows, BART runs inside the Windows Subsystem for Linux (WSL); the app calls it as `wsl bart`.

1. Install WSL. Start a Windows PowerShell as administrator and run `wsl --install`. This installs WSL 2 and Ubuntu; a restart is needed. The first time you start the Linux prompt you are asked to create a user account. See <https://pureinfotech.com/install-wsl-windows-11/> for older systems.

2. Update the Linux distribution:

```
sudo apt-get update
sudo apt-get dist-upgrade
```

3. Install the packages BART needs (gcc 12 or newer):

```
sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev libopenblas-dev gfortran
```

4. Download, build and install BART:

```
wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
tar xvfz v1.0.00.tar.gz
cd bart-1.0.00
make
sudo make install
```

5. Check from a Windows command prompt: `wsl bart version`

Elastix registration

Image registration uses elastix, which comes with T1mapp: version 5.3.1 on macOS (Apple silicon), 5.1.0 on Windows. When elastix cannot be run, T1mapp registers the images with its own built-in registration instead (translation, rigid or affine; a B-spline registration is then done as affine), and says so.



5. Running the software

Open **T1mapp** from the Applications folder (macOS), or from the Start menu or the desktop shortcut (Windows, provisional). At its start the app looks for elastix and BART, and the message box reports what it found; "Ready" says it is done. Only **Load MRD** and **Exit** are on until a scan is loaded.

Newer versions. At its start the app asks GitHub (<https://github.com/Moby1971/MRI-apps>) whether there is a newer release of it. The message box then shows the current and the newest version and, when there is a newer one, where to download it. Without a network connection it says that the version could not be checked, and the app works as usual. `checkForUpdates` in the settings file switches the check off; `versionCheckTimeout` sets how many seconds it waits for GitHub (5).

This manual. The **Manual** button at the bottom of the window, with the app's icon on a book, opens this manual in the computer's PDF viewer.

The window has a fixed size. On a screen too small for it, the window fits the screen and can be scrolled.

Settings and log files

T1mapp keeps its files in a folder of its own, which it makes at its first start:

macOS	<code>~/Library/Application Support/T1mapp/0.2.0p</code>
-------	--

Windows (provisional)	<code>%APPDATA%\T1mapp\0.2.0p</code>
-----------------------	--------------------------------------

The MATLAB app's settings and logs are never read or changed.

Settings. The settings are kept in `t1mappSettings.json`. The file is written with the standard values the first time a version starts, and can then be edited by hand; changes take effect at the next start. It holds the folders where loading and the exports start (`mrdImportPath`, `defaultExportPath`), the colour scheme, the sounds, the reordering of segmented k-space, the values the reconstruction, registration, fit and display fields start with (the colour maps of the three maps among them), and the number of log files kept. A value that cannot be used is replaced by its standard value, and the message box says so; a file that cannot be read at all is set aside as `t1mappSettings-unreadable-<time>.json` and replaced by a new one. The colour scheme and the sounds can also be switched in the App settings tab, for the session only.

Log files. Every session writes a log file, `t1mapp-log-<date and time>.txt`, to the same folder. The log keeps every message and the parameters of each import, reconstruction, registration and fit. Logs older than the newest 50 are removed at start.



6. Basic operation

The T1mapp program operates from a single window with 5 panels.

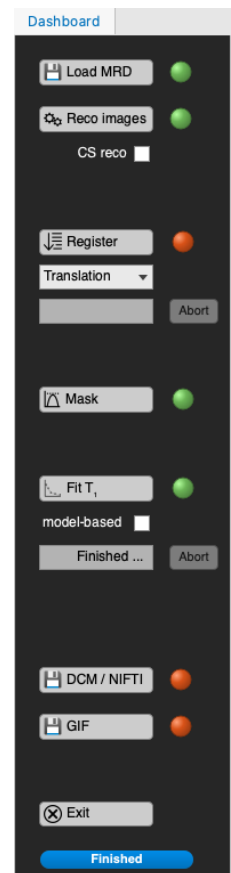
Panel 1: Dashboard. The task buttons and parameters that control the reconstruction process. The tasks run from top to bottom; registration and masking can be left out. A green light shows a task done, yellow a task running, and red one not done or failed. At the bottom of the Dashboard one progress bar shows the progress of every task ("Registration 40 %", "Fit 40 %", "Finished").

Panel 2: Raw images / Reco parameters / Fit parameters / App settings.

Panel 3: Sequence parameters. The relevant acquisition parameters; for inversion recovery the first and last TI can be edited, which spaces the TIs again and clears a fit.

Panel 4: T_1 map / M_0 & R^2 . During and after fitting, the T_1 maps. The M_0 and R^2 goodness-of-fit maps are displayed in the second tab.

Panel 5: Messages. The program messages, newest on top, and the status: OK, BUSY, WARNING or ERROR.



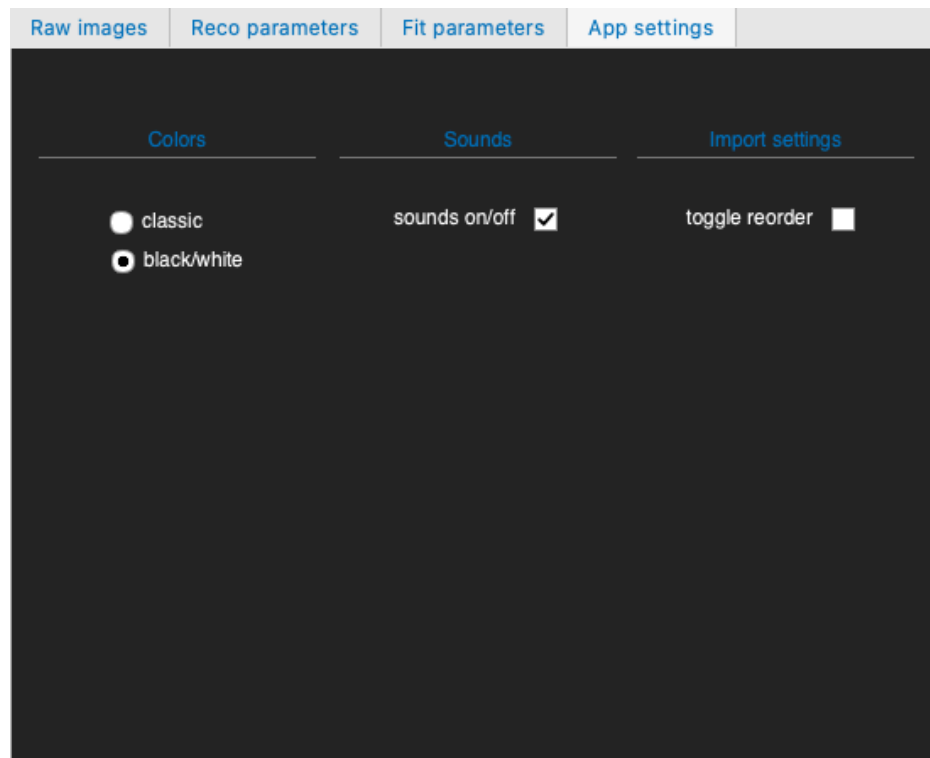
Sequence parameters					
datafile	3572_0.MRD			FOV (mm)	35
TR (ms)	10	last TR (ms)	10	#TRs	1
1st TI (ms)	100	last TI (ms)	4020	#TIs	50
#dynamics	1			#slices	1
aspect ratio	1.00			slice thickness (mm)	1
				dimx	128
				dimy	128

Step 1: Loading data

Press **Load MRD** to load the MRD raw data file.

T1mapp recognizes three T_1 measurements from the MRD parameters: inversion recovery (Look-Locker FLASH), saturation recovery (multi-TR) and variable flip angle series; other scans are refused, with the reason in the message box, and the scan loaded before stays. Retrospective's and P2ROUD's MRD exports load as well (a variable flip angle CINE from Retrospective, for instance).

If a segmented FLASH scan does not reconstruct properly, tick **toggle reorder** in the App settings tab and load it again.

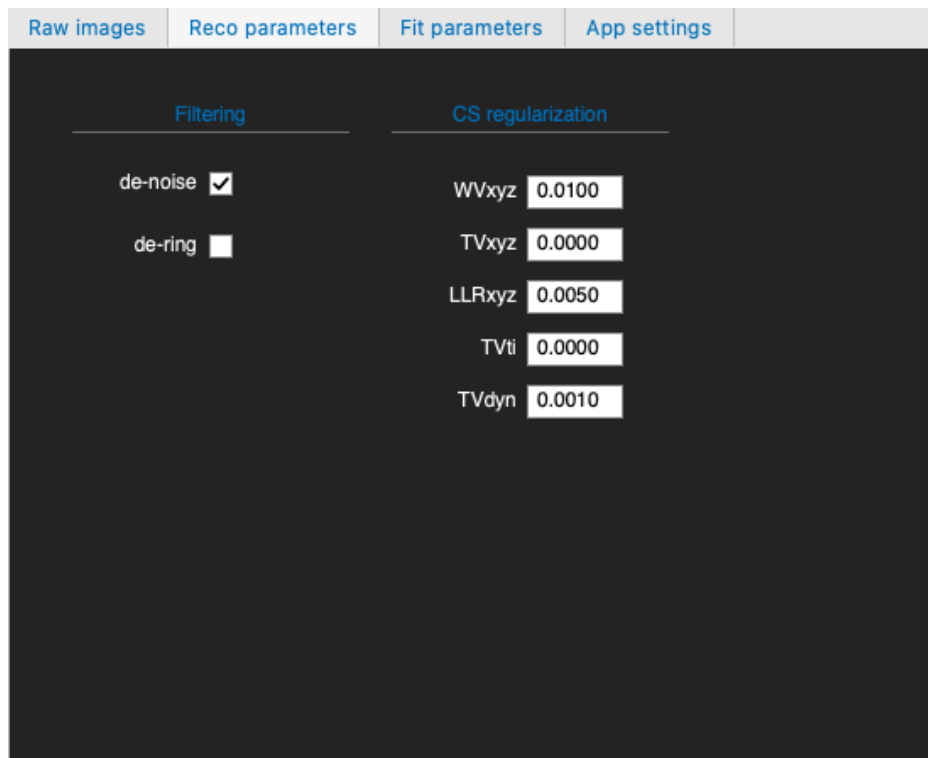


For multi-receiver coil data, select one of the MRD files; the app loads the others. The relevant acquisition parameters are shown in panel 3.

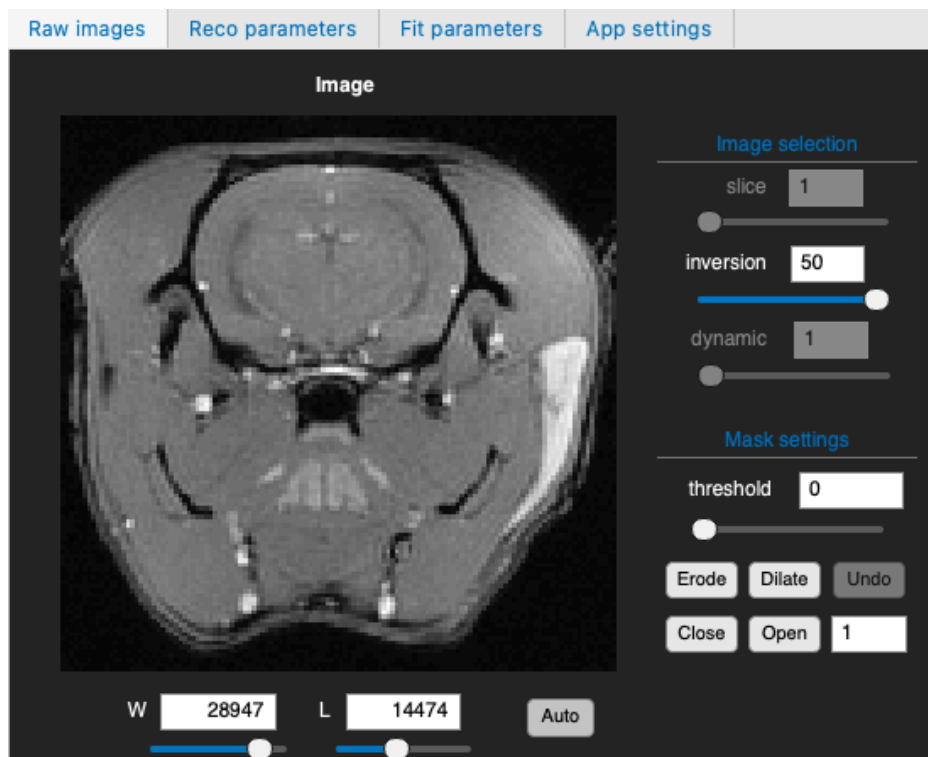
Step 2: Image reconstruction

Loading reconstructs the images already. Press **Reco images** to reconstruct them again, for instance with compressed sensing.

The standard reconstruction is a 2D or 3D FFT with PCA de-noising, on by default. When the **CS reco** checkbox is checked, the images are reconstructed by compressed sensing — with BART when it is available, else with the app's own sigpy engine; the message box says which — with the weights of the Reco parameters tab: wavelets (WV), total variation (TV) and locally low rank (LLR), in space (xyz) or along the series (TVti, TVtr or TVfr) and the dynamics (TVdyn or TVfa). That tab also switches PCA de-noising (**de-noise**) and Gibbs ringing suppression (**de-ring**, off by default).



Images with different inversion times and dynamics can be inspected with the sliders of the Raw images tab (**slice**, **inversion** — or **TR**, **frame** — and **dynamic** — or **flip angle**). The window and level are set with the **W** and **L** fields and sliders, or with **Auto**.



Step 3: Image registration

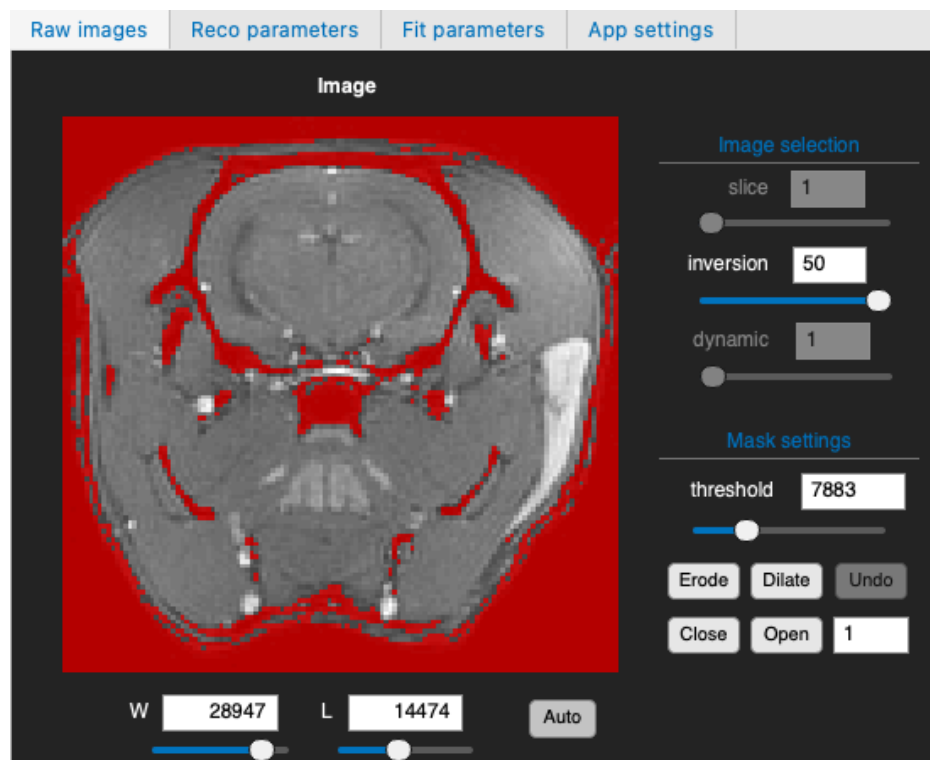
NOTE: this step usually can be omitted.

In case the images as a function of the inversion times or dynamics are not perfectly registered, press **Register** to register them to the last inversion time (the longest TR, or the brightest flip angle) with elastix, as a **Translation**, **Rigid**, **Affine** or **B-Spline** transform (the list below the button). **Abort** stops it. Registering clears the mask and the maps, and rules out the model-based fit.

Step 4: Masking the background

To prevent unnecessary fitting of the background pixels, the images need to be masked first; without a mask every pixel is fitted, the background included.

Press **Mask** to mask them with an automatic (Otsu) threshold per slice. The red pixels indicate the regions that will be skipped during T₁ fitting.



In case the automatic threshold is not satisfying, it can be adjusted per slice with the **threshold** field or slider. The mask can be refined by **Erode**, **Dilate**, **Close** and **Open** operations, of the size in pixels in the field beside them; **Undo** takes the last one back.

Step 5: Fitting T₁ maps

The Fit parameters tab holds the fit settings.



Raw images		Reco parameters		Fit parameters		App settings	
Slice selection first slice 1 last slice 1				Dynamic selection first dynamic 1 last dynamic 1		Goodness of fit min R-square 0.80	
Inversion time selection							
☒ 1	☒ 9	☒ 17	☒ 25	☒ 33	☒ 41	☒ 49	☒ 57
☒ 2	☒ 10	☒ 18	☒ 26	☒ 34	☒ 42	☒ 50	☒ 58
☒ 3	☒ 11	☒ 19	☒ 27	☒ 35	☒ 43	☒ 51	☒ 59
☒ 4	☒ 12	☒ 20	☒ 28	☒ 36	☒ 44	☒ 52	☒ 60
☒ 5	☒ 13	☒ 21	☒ 29	☒ 37	☒ 45	☒ 53	☒ 61
☒ 6	☒ 14	☒ 22	☒ 30	☒ 38	☒ 46	☒ 54	☒ 62
☒ 7	☒ 15	☒ 23	☒ 31	☒ 39	☒ 47	☒ 55	☒ 63
☒ 8	☒ 16	☒ 24	☒ 32	☒ 40	☒ 48	☒ 56	☒ 64
☒ 9	☒ 17	☒ 25	☒ 33	☒ 41	☒ 49	☒ 57	☒ 65
☒ 10	☒ 18	☒ 26	☒ 34	☒ 42	☒ 50	☒ 58	☒ 66
☒ 11	☒ 19	☒ 27	☒ 35	☒ 43	☒ 51	☒ 59	☒ 67
☒ 12	☒ 20	☒ 28	☒ 36	☒ 44	☒ 52	☒ 60	☒ 68
☒ 13	☒ 21	☒ 29	☒ 37	☒ 45	☒ 53	☒ 61	☒ 69
☒ 14	☒ 22	☒ 30	☒ 38	☒ 46	☒ 54	☒ 62	☒ 70
☒ 15	☒ 23	☒ 31	☒ 39	☒ 47	☒ 55	☒ 63	☒ 71
☒ 16	☒ 24	☒ 32	☒ 40	☒ 48	☒ 56	☒ 64	☒ 72
☒ 17	☒ 25	☒ 33	☒ 41	☒ 49	☒ 57	☒ 65	☒ 73
☒ 18	☒ 26	☒ 34	☒ 42	☒ 50	☒ 58	☒ 66	☒ 74
☒ 19	☒ 27	☒ 35	☒ 43	☒ 51	☒ 59	☒ 67	☒ 75
☒ 20	☒ 28	☒ 36	☒ 44	☒ 52	☒ 60	☒ 68	☒ 76
☒ 21	☒ 29	☒ 37	☒ 45	☒ 53	☒ 61	☒ 69	☒ 77
☒ 22	☒ 30	☒ 38	☒ 46	☒ 54	☒ 62	☒ 70	☒ 78
☒ 23	☒ 31	☒ 39	☒ 47	☒ 55	☒ 63	☒ 71	☒ 79
☒ 24	☒ 32	☒ 40	☒ 48	☒ 56	☒ 64	☒ 72	☒ 80
☒ 25	☒ 33	☒ 41	☒ 49	☒ 57	☒ 65	☒ 73	☒ 81
☒ 26	☒ 34	☒ 42	☒ 50	☒ 58	☒ 66	☒ 74	☒ 82
☒ 27	☒ 35	☒ 43	☒ 51	☒ 59	☒ 67	☒ 75	☒ 83
☒ 28	☒ 36	☒ 44	☒ 52	☒ 60	☒ 68	☒ 76	☒ 84
☒ 29	☒ 37	☒ 45	☒ 53	☒ 61	☒ 69	☒ 77	☒ 85
☒ 30	☒ 38	☒ 46	☒ 54	☒ 62	☒ 70	☒ 78	☒ 86
☒ 31	☒ 39	☒ 47	☒ 55	☒ 63	☒ 71	☒ 79	☒ 87
☒ 32	☒ 40	☒ 48	☒ 56	☒ 64	☒ 72	☒ 80	☒ 88
☒ 33	☒ 41	☒ 49	☒ 57	☒ 65	☒ 73	☒ 81	☒ 89
☒ 34	☒ 42	☒ 50	☒ 58	☒ 66	☒ 74	☒ 82	☒ 90
☒ 35	☒ 43	☒ 51	☒ 59	☒ 67	☒ 75	☒ 83	☒ 91
☒ 36	☒ 44	☒ 52	☒ 60	☒ 68	☒ 76	☒ 84	☒ 92
☒ 37	☒ 45	☒ 53	☒ 61	☒ 69	☒ 77	☒ 85	☒ 93
☒ 38	☒ 46	☒ 54	☒ 62	☒ 70	☒ 78	☒ 86	☒ 94
☒ 39	☒ 47	☒ 55	☒ 63	☒ 71	☒ 79	☒ 87	☒ 95
☒ 40	☒ 48	☒ 56	☒ 64	☒ 72	☒ 80	☒ 88	☒ 96

The first and last slice and dynamic (for variable flip angle data, frame) that will be fitted can be selected.

min R-square is the lowest R^2 accepted: where the fit of a pixel has a lower R^2 (0.80 by default), or gives a T_1 above 10 s, the T_1 and M_0 values are set to zero.

The bottom part of this tab lists the points of the series in the order of the scan (inversion times, TRs or flip angles): those ticked are fitted, so points can be left out, as long as 3 remain.

Press **Fit T_1** to start the T_1 fitting; **Abort** on the Dashboard stops it. The maps fill in slice by slice while the fit runs.

For inversion recovery the fit is a least-squares fit of the equation

$$Signal = |A - B \exp(-TI / T_1^*)|, T_1 = (B / A - 1) T_1^*$$

Model-based fitting

Tick **model-based** to fit a model-based reconstruction with BART: the T_1 map is then fitted to the k-space directly. It is offered for 2D inversion recovery that has not been registered, when BART is available.

Step 6: Fit results

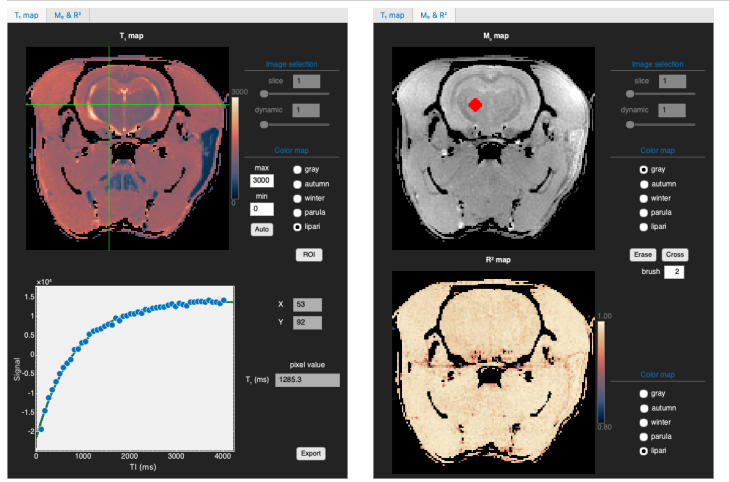
The T_1 maps are shown in the T_1 map tab, on the scale set with **max** and **min**, or with **Auto**. An M_0 map and an R^2 map are shown in the second tab. A different colour map can be chosen for each (gray, autumn, winter, parula, lipari). After the fit, the T_1 scale runs from 0 to twice the median T_1 , and the sliders beside the maps step through their slices and dynamics.



The colour bars stand beside the T_1 and R^2 maps, with their end values above and below them: the scale's max and min for T_1 , 1 and the R^2 cut-off for R^2 .

T_1 map

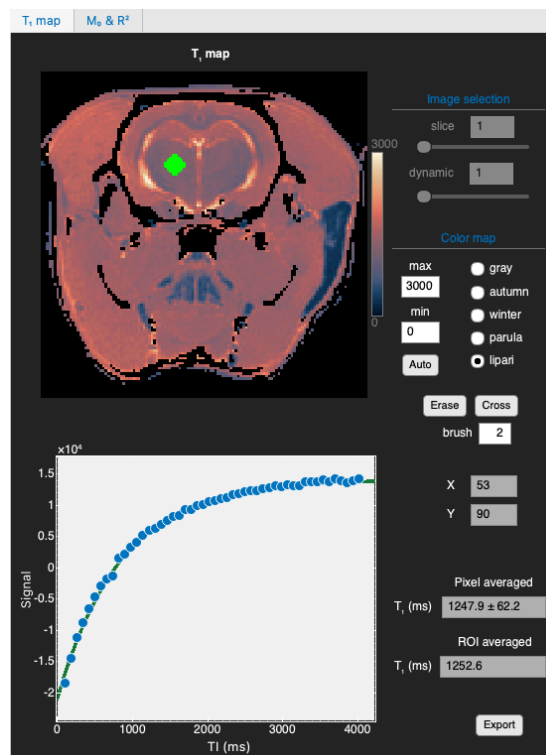
M_0 & R^2



Step 7: T_1 values of pixels and ROIs

When you click on the T_1 map a cross-hair is shown, with the signal of that pixel as a function of TI (symbols), the fitted curve (line) and its T_1 value (**pixel value**). The signal of an inversion recovery is shown with the sign the fit restored. A red symbol indicates that this point was left out of the fit, according to the list in the Fit parameters tab (see step 5).

You can toggle between the cross-hair and region-of-interest (ROI) values with the **ROI / Cross** button. A single ROI can be 'painted' on the T_1 map, in one or more slices, by mouse clicks on the image, with the brush size in pixels set below the button.



For the ROI, two T_1 values are calculated:

- **Pixel averaged** — the average of the T_1 values of the pixels, with their spread: $T_1(\text{ROI}) = 1 / \text{mean}(R_1 \text{ of the pixels})$.
- **ROI averaged** — the T_1 equation is fitted to the average signal intensities in the ROI, producing a single T_1 value for that ROI.

The ROI can be removed with **Erase**. The ROI-averaged signal is written to a CSV file with **Export**, `roiData-<tag>.csv`, beside the inversion times, TRs or flip angles, in a folder you choose.

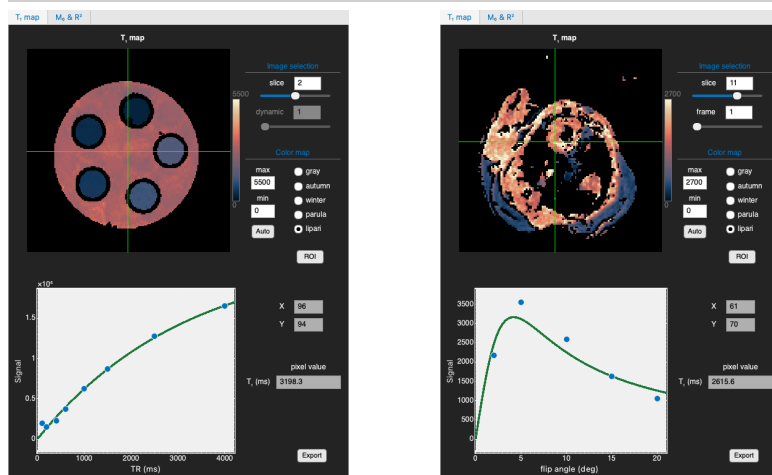
Step 8: Saturation recovery and variable flip angle

The app also analyzes saturation recovery data. The data need to contain an array of TR values. T_1 is obtained by fitting the signal intensity as a function of TR to

$$\text{Signal} = A (1 - \exp(-TR / T_1))$$

Variable flip angle series (spoiled gradient echo, the flip angles of the scan's VFA array) are fitted per pixel as a straight line through $S / \sin \alpha$ against $S / \tan \alpha$ (DESPOT1), whose slope is $\exp(-TR / T_1)$. For these series and for saturation recovery, the sliders and the selection lists are named after the TRs or the flip angles; flip angle series are fitted per cardiac frame, without B1 correction.

Saturation recovery (phantom) Variable flip angle (mouse heart)



Step 9: Exporting the maps

There are two ways to export the T_1 , M_0 and R^2 maps.

1. **DCM / NIFTI** — exports the maps in DICOM and NIfTI format for further processing in third-party software, and the T_1 map also as a BIDS quantitative map (`BIDS/derivatives/t1map/sub-<tag>/anat/ sub-<tag>_T1map.nii.gz`, in seconds; the DICOM and NIfTI maps are in ms). The program looks for the scanner's DICOM files of the scan and takes their header; if they are not found, the header, with the image position and orientation, is made from the MRD file.
2. **GIF** — exports the T_1 , M_0 and R^2 maps as animated GIF files, with two pictures of the app's window.



When the MRD file lies in the scanner's folders, .../MRD/<study>/<scan>, the exports go to .../Data/<study>/DICOM, Nifti, BIDS and GIF, each into a new numbered folder; otherwise the app asks for a folder. The ROI data of step 7 go to a folder of your choice.

Step 10: Exit

Press **Exit** to shut down the program. Closing the window does the same, also while a step runs: the registration or fit is stopped, the window disappears, and the program closes as soon as the step has ended.



7. Differences from the MATLAB app

T1mapp follows the MATLAB app T1mapp: the same window, controls and results. Where it differs:

- No MATLAB, MATLAB Runtime or toolboxes are needed; elastix and dcm2niix are inside the app.
- Without BART, compressed sensing runs with the app's own sigpy engine; only the model-based fit needs BART.
- When elastix cannot be run, the built-in registration replaces MATLAB's imregister.
- Settings and logs are in a folder of its own (.../T1mapp/0.2.0p).
- One progress bar, at the bottom of the Dashboard, for every task; the colour bars beside the maps; T_1 and M_0 written with subscripts; the fit plot with a dark green curve.

8. Credits

- **BART toolbox** — <https://mrirecon.github.io/bart/>. M. Uecker et al., Berkeley Advanced Reconstruction Toolbox, Proc. Intl. Soc. Mag. Reson. Med. 23, 2486 (2015).
- **sigpy** — Frank Ong and Michael Lustig, SigPy: a python package for high performance iterative reconstruction, Proc. Intl. Soc. Mag. Reson. Med. 27, 4819 (2019).
- **elastix** — S. Klein, M. Staring, K. Murphy, M.A. Viergever, J.P.W. Pluim, elastix: a toolbox for intensity based medical image registration, IEEE Transactions on Medical Imaging 29(1), 196–205 (2010). <https://elastix.dev>
- **PCA denoising** — J. Veraart, D. S. Novikov, D. Christiaens, B. Ades-Aron, J. Sijbers, E. Fieremans, Denoising of diffusion MRI using random matrix theory, NeuroImage 142, 394–406 (2016).
- **Gibbs ringing suppression** — Kellner, E., Dhital, B., Kiselev, V.G. and Reiser, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med 76(5), 1574–1581 (2016).
- **DESPOT1** — S.C.L. Deoni, B.K. Rutt, T.M. Peters, Rapid combined T1 and T2 mapping using gradient recalled acquisition in the steady state, Magn Reson Med 49(3), 515–526 (2003).
- **Colour maps** — F. Crameri, Scientific colour maps (lipari), Zenodo, doi:10.5281/zenodo.1243862.
- **NIfTI conversion** — dcm2niix: C. Rorden et al., <https://github.com/rordenlab/dcm2niix>.